

UnifAI / Raksha Enterprise AI Platform

Complete Documentation Suite & Master Directory Index

Classification: Enterprise System Documentation

Suite Release: Version 2.4.0

Storage Directory: [/pdf/](#) & [/docs/pdf/](#)

Generated Date: October 2026

1. Documentation Suite Overview

This documentation folder ([pdf/](#)) contains the complete technical, operational, architectural, and security manuals for the **UnifAI Enterprise AI Gateway** and **Raksha Browser Guard** endpoint protection suite.

All documentation is provided in both **Markdown (.md)** format for source tracking and **Adobe PDF (.pdf)** format for distribution, printing, and archiving.

2. Document Catalog & Purpose

#	Document File	Formats Available	Primary Audience	Description & Coverage
1	Technical Documentation	01_TECHNICAL_DOCUMENTATION.md 01_Technical_Documentation.pdf	Architects, Lead Developers, Security Engineers	Complete system architecture, Fasthttp gateway engine, GORM data models, ~28 AI provider connectors, Browser Guard MitM proxy architecture, DLP engine, RBAC matrix, and plugin system.
2	Server Implementation Guide	02_SERVER_IMPLEMENTATION_GUIDE.md 02_Server_Implementation_Guide.pdf	DevOps, SRE, Systems Administrators	Hardware capacity planning, bare-metal Linux systemd deployment, complete .env configuration breakdown, Nginx & Caddy reverse proxy setup, Prometheus metrics, health checks, log management, and zero-downtime rolling upgrades.

#	Document File	Formats Available	Primary Audience	Description & Coverage
3	Admin Console Guide	03_ADMIN_CONSOLE_GUIDE.md 03_Admin_Console_Guide.pdf	Platform Admins, InfoSec, Workspace Managers	First-time bootstrap setup, User lifecycle management, Custom RBAC role builder (33 resources × 6 operations), Access Profiles, Model Provider connections, Virtual Key spend limits, Browser AI Target Websites, DLP Rules, Fleet Agent monitoring, and Workspace Configuration views (Client Settings, Compatibility, Caching, Security, API Keys, Performance Tuning, Feature Flags).
4	End-User & Developer Guide	04_USER_GUIDE.md 04_User_Guide.pdf	Employees, Developers, Data Scientists	Account registration and approval, navigating the Prompt Repository, Skills Repository, Model Playground, Developer API integration (Python, Node.js, LangChain, Cursor, Claude Desktop), Desktop Browser Guard installation for Windows & macOS, in-browser DLP alerts, and privacy protections.
5	Database Configuration Guide	05_DATABASE_CONFIGURATION_GUIDE.md 05_Database_Configuration_Guide.pdf	DBAs, DevOps, Infrastructure Engineers	PostgreSQL 16 setup on Ubuntu/Debian/RHEL/Docker, UTF-8 schemas, pg_hba.conf network security, SSL/TLS database encryption, GORM table schemas, high-volume log partitioning, PgBouncer connection pooling, production performance tuning parameters, and automated daily backup scripts with disaster recovery restore procedures.

#	Document File	Formats Available	Primary Audience	Description & Coverage
6	Docker & SSL/TLS Configuration Guide	06_DOCKER_AND_SSL_CONFIGURATION_GUIDE.md 06_Docker_and_SSL_Configuration_Guide.pdf	DevOps, Cloud Engineers, SREs, SecOps	Multi-container Docker Compose architecture, exhaustive Docker command reference (up, down, logs, exec, build, prune), Public HTTPS with Let's Encrypt / Certbot, OpenSSL self-signed certificate generation, Browser Guard Root CA generation (raksha-ca.crt), client trust store installation (Windows certutil, macOS security, Linux update-ca-certificates, Firefox NSS), Nginx reverse proxy configuration, and SSL handshake diagnostic commands.

3. Quick Reference: Critical System Ports

- Public Web / HTTPS: 443 (Nginx / Caddy Reverse Proxy)
- Plain HTTP Redirect: 80 (Certbot ACME challenge & HTTPS redirect)
- UnifAI Go Backend Core: 6000 (Internal localhost bind: 127.0.0.1:6000)
- PostgreSQL Database: 5432 (or 6432 with PgBouncer)
- Local Ollama LLM Service: 11434 (Internal service for AI Guard Bot)
- Browser Guard Local MitM Proxy: 18103 (Bound to client laptop loopback 127.0.0.1)
- Browser Guard Local PAC Server: 18195 (Bound to client laptop loopback 127.0.0.1)

UnifAI Documentation Suite — Built for Enterprise AI Governance & Security.

UnifAI / Raksha Enterprise AI Gateway & Guard

Technical Architecture & Specification Document

Document Version: 2.4.0

Target Systems: UnifAI Core, Raksha HTTP Gateway, Browser Guard Fleet, Management UI

Classification: Enterprise Technical Documentation

Target Audience: Software Architects, Lead Developers, Security Engineers, DevOps Engineers

1. Executive Summary & High-Level System Overview

UnifAI (incorporating the Raksha AI Security Suite) is an enterprise-grade AI Gateway, Governance Platform, and Endpoint AI Guardrail system. It addresses two critical enterprise AI challenges in a unified architecture:

- 1. Centralized AI Gateway & Governance (Server Plane):** Provides a unified, high-performance API gateway that unifies over 28 commercial, open-source, and private AI providers (OpenAI, Anthropic, Google Gemini, AWS Bedrock, Ollama, Cohere, DeepSeek, Groq, Mistral, etc.) under an OpenAI-compatible interface with virtual key management, budget enforcement, data-loss prevention (DLP), semantic caching, prompt governance, and Model Context Protocol (MCP) tool routing.
- 2. Endpoint AI Guard & Browser DLP (Client Plane):** Deploys a zero-bypass endpoint daemon (Raksha Browser Guard) on managed employee workstations (Windows and macOS). The guard transparently intercepts web-based interactions with consumer and enterprise generative AI websites (ChatGPT, Claude.ai, Gemini, Perplexity, DeepSeek, Copilot, etc.), enforcing real-time prompt redaction, sensitive file upload blocking, regulatory compliance policies, and automated audit logging.

[illegible]

2. Core Architectural Pillars

2.1 Dual-Plane Design

The server runs both planes inside a single high-performance Go executable compiled with Go 1.26+ and **valyala/fasthttp**:

- **Data Plane** (**/v1/***, **/openai/***, **/anthropic/***, **/genai/***): Handles ultra-low latency AI request proxying, streaming token aggregation, prompt transformations, caching, and rate limiting. Designed for sub-millisecond gateway overhead.
- **Control Plane** (**/api/***, **/ws**, **/health**): Provides the administrative backend, user identity lifecycle, role-based access control (RBAC), telemetry dashboards, MCP tool registry, and Browser AI agent control plane.

2.2 Endpoint Protection (Browser Guard)

- **Local Proxy Architecture**: Runs a local Python 3.11+ mitmproxy-based engine bound to loopback (**127.0.0.1:18103**) paired with a lightweight PAC (Proxy Auto-Configuration) daemon (**127.0.0.1:18195**).
- **Targeted Interception**: Network traffic to non-AI destinations is bypassed completely with zero latency impact. Only configured target websites (ChatGPT, Claude, etc.) route through the local inspection hook.
- **Fail-Closed / Fail-Open Modes**: Configurable behavior in enterprise policies. In strict compliance environments (**RAKSHA_FAIL_OPEN=0**), traffic to monitored targets is blocked if the guard daemon is halted.
- **DLP Engine**: Evaluates regex pattern dictionaries, keyword rules, and an optional local Ollama AI Guard Bot for semantic evaluation of prompt content before requests leave the employee workstation.

3. Technology Stack Reference

Subsystem	Technology / Library	Version / Baseline	Justification / Role
Server Runtime	Go (Golang)	1.26.4+	Memory efficiency, concurrent goroutine scheduling, zero-cost garbage collection tuning.
HTTP Engine	valyala/fasthttp	v1.58+	Zero-allocation HTTP server engine capable of handling 100k+ req/sec.
Routing	fasthttp/router	v1.5+	High-performance radix-tree HTTP request routing.
ORM / Data Access	GORM	v1.25+	Type-safe database abstraction across PostgreSQL, MySQL, and SQLite.
Primary Database	PostgreSQL	14.x / 15.x / 16.x	ACID compliance, JSONB support, partitioned log storage, and robust concurrency.
Client UI	React, TypeScript, Vite	React 19, TS 5.9, Vite 8	Enterprise single-page administrative dashboard.
State & UI Kit	Redux Toolkit, Radix UI, Tailwind	Tailwind CSS 4, Radix Primitives	Accessible UI components, responsive layout, dark/light theme management.
Guard Runtime	Python (Bundled Standalone)	3.11+	Mitmproxy addon framework, PyInstaller self-contained binary packaging.

Subsystem	Technology / Library	Version / Baseline	Justification / Role
Windows Packaging	Inno Setup	6.x	Automated Windows installer packaging with silent install flags and auto-update support.
macOS Packaging	Apple PackageMaker / pkgbuild	macOS Sonoma/Sequoia compatible	LaunchAgent deployment and system keychain root CA integration.
Containerization	Docker & Docker Compose	Docker 24+, Compose v2	Multi-stage image build and isolated multi-service orchestration.

4. Repository Structure & Module Breakdown

The codebase is organized into distinct Go modules, frontend applications, and endpoint agents:

```
unifai_project/ ■■■■ core/ # Core AI provider engine ■ ■■■■ providers/ # ~28 provider implementations ■ ■ ■■■■ openai/ # OpenAI Chat, Audio, Images, Embeddings ■ ■ ■■■■ anthropic/ # Anthropic Claude Messages API ■ ■ ■■■■ google/ # Google Gemini Pro / Flash SDK ■ ■ ■■■■ bedrock/ # AWS Bedrock Converse and InvokeModel ■ ■ ■■■■ ollama/ # Local Ollama client implementation ■ ■ ■■■■ azure/ # Azure OpenAI Service integration ■ ■■■■ schemas/ # Universal request/response schemas ■ ■■■■ mcp/ # Model Context Protocol client & code mode ■ ■■■■ keyselectors/ # Round-robin, weighted, and lowest-latency selection ■ ■■■■ network/ # HTTP client pool, retry policies, backoff ■ ■■■■ framework/ # Shared infrastructure modules ■ ■■■■ configstore/ # GORM database models & repository queries ■ ■ ■■■■ tables/ # GORM schema definitions for 40+ tables ■ ■■■■ logstore/ # LLM and Browser Guard request/response logs ■ ■■■■ rbac/ # 33 Resources × 6 Operations access engine ■ ■■■■ encrypt/ # AES-256-GCM symmetric database encryption ■ ■■■■ cluster/ # Multi-node cache and state replication ■ ■■■■ loadbalancer/ # Gateway load distribution algorithms ■ ■■■■ circuitbreaker/ # Upstream provider fault tolerance ■ ■■■■ featureflags/ # Dynamic runtime feature gating ■ ■■■■ plugins/ # Interceptor plugin implementations ■ ■■■■ guardrails/ # Prompt and response safety validation ■ ■■■■ semanticcache/ # Vector/hash-based response caching ■ ■■■■ telemetry/ # OpenTelemetry, Prometheus metrics exporter ■ ■■■■ governance/ # Token budget enforcement and rate limits ■ ■■■■ maxim/ # Prompt evaluation and observability ■ ■■■■ transports/ ■ ■■■■ raksha-http/ # Server endpoint and HTTP controllers ■ ■■■■ main.go # Binary startup, flags, environment loading ■ ■■■■ server/ # Fasthttp server initialization & middleware ■ ■■■■ handlers/ # 50+ API controller implementations ■ ■ ■■■■ auth.go # Login, session cookies, password resets ■ ■ ■■■■ browser_ai.go # Browser Guard telemetry and policies ■ ■ ■■■■ browser_ai_download.go # Endpoint installer packaging and distribution ■ ■ ■■■■ virtual_keys.go # Virtual key creation, budgets, and tokens ■ ■ ■■■■ models.go # Model catalog and provider management ■ ■ ■■■■ rbac.go # Roles, permissions, and access profiles ■ ■■■■ integrations/ # Pass-through provider route handlers ■ ■■■■ apps/browser-guard/ # Desktop Browser Guard client ■ ■■■■ agent/ # System tray, PAC daemon, and updater ■ ■ ■■■■ guard_bootstrap.py # Process lifecycle and cert verification ■ ■ ■■■■ guard_pac_server.py # Dynamic PAC file generator and server ■ ■ ■■■■ guard_tray.py # Cross-platform system tray status menu ■ ■■■■ proxy/ # Mitmproxy interception addon ■ ■ ■■■■ browser_ai_proxy.py # Traffic inspection, DLP rules, and redaction ■ ■ ■■■■ cert_installer.py # Automated Root CA installation in OS stores ■ ■■■■ installer/ # Windows Inno Setup script & macOS build scripts ■ ■■■■ release/ # Pre-built binaries, updaters, and VERSION.txt ■ ■■■■ ui/ # Single Page Application (React 19 / Vite) ■ ■■■■ app/ # Route structure & workspace views ■ ■ ■■■■ workspace/ # Authenticated admin & user dashboard ■ ■■■■ lib/ # Redux slices, RTK Query API clients ■ ■■■■ public/ # Logos, static assets, and favicon ■ ■■■■ configs/ # Configuration templates and model parameters ■■■■ deploy/ # Dockerfiles, docker-compose, and Helm charts ■■■■ docs/ # Project documentation & PDF artifacts
```

5. Security & Authentication Architecture

5.1 Dual-Token Authentication

- 1. **Dashboard Sessions (Browser):** Authenticated using HTTP-Only, Secure, SameSite cookies carrying encrypted session tokens. Protected against cross-site scripting (XSS) and cross-site request forgery (CSRF).

2. **API Access (Developer / CI/CD)**: Authenticated via HTTP Authorization header: **Authorization: Bearer <VIRTUAL_KEY>**. Virtual keys are cryptographically generated 48-character strings tied to specific workspaces, rate limits, and token budgets.

5.2 Role-Based Access Control (RBAC) Matrix

The platform implements a multi-tier permission matrix combining:

- **System Roles**:
- **admin**: Global administrator. Full access across all 33 resources and system settings.
- **sub_admin**: Department / workspace administrator. Manages keys, policies, prompts, and Browser AI, but cannot alter master provider credentials or core infrastructure settings.
- **user**: Standard employee. Read-only access restricted strictly to assigned sections (e.g., Prompt Repository, Personal Logs).
- **Custom Roles**: Granular control across 33 distinct system resources (**VirtualKeys**, **ModelProvider**, **GuardrailRules**, **Logs**, **Users**, etc.) mapped against 6 discrete operations (**Create**, **Read**, **Update**, **Delete**, **View**, **Download**).
- **Data Access Control (DAC)**: Scopes records to **all-data** (organization-wide) or **own-data** (records created by or assigned to the authenticated user).

5.3 Cryptography & Secrets at Rest

All sensitive secrets—including third-party provider API keys, SMTP passwords, and external database credentials—are encrypted before storage in PostgreSQL using AES-256-GCM. The encryption key is derived from

UNIFAI_ENCRYPTION_KEY in **.env**.

5.4 Endpoint Guard Fleet Authentication

To prevent rogue network agents from polluting security telemetry or downloading enterprise DLP policies, the Browser Guard fleet uses a shared fleet secret (**RAKSHA_GUARD_SECRET**).

- Agents must send **X-Raksha-Guard-Secret: <SECRET>** on every heartbeat, policy fetch, and log upload.
- When **RAKSHA_GUARD_REQUIRE_SECRET=1**, the gateway enforces fail-closed rejection (HTTP 401/403) for any request lacking the valid secret.

6. Browser Guard Interception & Data Loss Prevention Engine

6.1 PAC (Proxy Auto-Configuration) Mechanism

The endpoint agent runs a lightweight HTTP server on **127.0.0.1:18195** that serves a dynamic PAC script to the local operating system:

```
function FindProxyForURL(url, host) { // Only route designated Generative AI domains to the local inspection proxy
  if (dnsDomainIs(host, "chatgpt.com") || dnsDomainIs(host, "claude.ai") || dnsDomainIs(host, "gemini.google.com") || dnsDomainIs(host, "deepseek.com") || dnsDomainIs(host, "perplexity.ai")) {
    return "PROXY 127.0.0.1:18103; DIRECT";
  } // All other enterprise and personal web traffic bypasses the proxy
  completely return "DIRECT";
}
```

6.2 Inspection & Rule Evaluation Pipeline

When an intercepted request hits the local proxy (**browser_ai_proxy.py**):

1. **Target Identification**: Matches hostname and URL path against downloaded enterprise **target_websites**.

2. **Body Extraction:** Extracts user prompt text and uploaded file attachments from multipart form-data or JSON payloads.
 3. **Rule Matching:** Evaluates active **guard_rules** in priority order:
 - **Regex Patterns:** Credit card numbers (Luhn algorithm), Social Security Numbers (SSN), API keys/tokens, AWS secrets, internal IP addresses.
 - **Keyword Dictionaries:** Confidential project codenames, source code markers, restricted customer names.
 - **AI Guard Bot:** Optional asynchronous semantic evaluation via local Ollama LLM.
 4. **Action Execution:**
 - **BLOCK:** Terminates request immediately, returning an HTTP 403 response with an enterprise branded violation message.
 - **REDACT:** Replaces sensitive matches with masking tokens (e.g., **[REDACTED_CREDIT_CARD]**) before forwarding upstream.
 - **WARN:** Allows request but logs a high-severity compliance alert to the management console and triggers email alerts.
 - **ALLOW:** Passes request unmodified and records standard telemetry.
-

7. Performance & Scalability Characteristics

- **Throughput:** Capable of proxying 25,000+ requests per second per server instance with under 1.2ms added latency overhead.
 - **Streaming Latency:** Server-Sent Events (SSE) chunks from upstream AI providers are forwarded directly to clients using zero-buffering fasthttp chunked transfers. Time-to-first-token (TTFT) degradation is below 2 milliseconds.
 - **Resource Footprint:**
 - Server Binary: ~45MB compiled executable. Idle RAM: ~65MB. Under full load: ~250MB - 1GB depending on cache size.
 - Endpoint Guard: Standalone Python daemon. Idle RAM: ~40MB. CPU utilization during active browsing: < 0.5%.
-

End of Technical Architecture Specification Document.

UnifAI / Raksha Enterprise Server Implementation Guide

Comprehensive Production Deployment & Operations Manual

Document Version: 2.4.0

Target Systems: Ubuntu 22.04/24.04 LTS, RHEL 9, Debian 12, Docker Engine 24+

Classification: Enterprise Infrastructure & Systems Administration

Target Audience: DevOps Engineers, Site Reliability Engineers (SRE), IT Infrastructure Leads

1. System Requirements & Capacity Planning

1.1 Hardware Specifications

Workload Tier	Concurrent Users	Daily Requests	Recommended CPU	Recommended RAM	Storage (SSD NVMe)
Starter / PoC	1 – 50	< 10,000	2 vCPU	4 GB	25 GB
Mid-Market	50 – 500	10k – 200,000	4 vCPU	8 – 16 GB	100 GB
Enterprise Fleet	500 – 5,000+	200k – 2M+	8 – 16 vCPU	32 – 64 GB	500 GB+

1.2 Operating System & Kernel Requirements

- Supported Linux Distributions:** Ubuntu 22.04 LTS / 24.04 LTS (Recommended), RHEL 9.x, Rocky Linux 9, Debian 12.
- Kernel Tuning:**
 - File descriptor limit (**fs.file-max**): `>= 65536`
 - Conntrack table size (**net.netfilter.nf_conntrack_max**): `>= 131072`
 - Ephemeral port range (**net.ipv4.ip_local_port_range**): `1024 65535`
 - TCP keepalive settings: **tcp_keepalive_time = 300, tcp_keepalive_intvl = 15, tcp_keepalive_probes = 5**

1.3 Network & Firewall Port Allocation

Port	Protocol	Default Bind	Exposure	Purpose
80	TCP	0.0.0.0	Public Internet	HTTP redirect to HTTPS (Certbot / Nginx)
443	TCP	0.0.0.0	Public Internet	HTTPS public endpoint (Reverse Proxy)
6000	TCP	127.0.0.1	Localhost / Internal	UnifAI Go Backend Core Service (APP_PORT)

Port	Protocol	Default Bind	Exposure	Purpose
18182	TCP	0.0.0.0	LAN / VPN Only	Docker Network Proxy (Optional profile network-proxy)
5432	TCP	127.0.0.1	Localhost / Private	PostgreSQL Database Service
11434	TCP	127.0.0.1	Localhost / Internal	Ollama Local LLM Server (Optional AI Guard Bot)
18103	TCP	127.0.0.1	Client Laptop Only	Browser Guard Local MitM Proxy (Bound to loopback)
18195	TCP	127.0.0.1	Client Laptop Only	Browser Guard Local PAC Daemon (Bound to loopback)

2. Server Installation: Bare-Metal / Systemd Deployment

For high-security or bare-metal enterprise environments running without Docker containers, UnifAI can be deployed directly as a native Linux systemd daemon.

2.1 Directory Structure Setup

Execute as **root** or an administrative user with **sudo**:

```
# Create dedicated unifai system user and group sudo useradd -r -s /bin/false -d /opt/unifai unifai # Create application directories sudo mkdir -p /opt/unifai/{bin,configs,data,logs,release} sudo chown -R unifai:unifai /opt/unifai sudo chmod 750 /opt/unifai
```

2.2 Installing the Binary & Assets

```
# Copy compiled server executable to binary path sudo cp transports/raksha-http/unifai-server /opt/unifai/bin/unifai-server sudo chmod 755 /opt/unifai/bin/unifai-server # Copy default configs and branding assets sudo cp -r configs/* /opt/unifai/configs/ sudo cp -r apps/browser-guard/release/* /opt/unifai/release/ sudo chown -R unifai:unifai /opt/unifai
```

2.3 Systemd Service Definition

Create the unit file at **/etc/systemd/system/unifai.service**:

```
[Unit] Description=UnifAI Enterprise AI Gateway & Guard Service After=network.target postgresql.service Wants=postgresql.service [Service] Type=simple User=unifai Group=unifai WorkingDirectory=/opt/unifai EnvironmentFile=/opt/unifai/.env ExecStart=/opt/unifai/bin/unifai-server Restart=always RestartSec=5s LimitNOFILE=65536 LimitNPROC=4096 # Sandboxing and security hardening ProtectSystem=full ProtectHome=true NoNewPrivileges=true PrivateTmp=true # Standard log routing to journald StandardOutput=journal StandardError=journal [Install] WantedBy=multi-user.target
```

Reload systemd and enable the service:

```
sudo systemctl daemon-reload sudo systemctl enable unifai.service sudo systemctl start unifai.service sudo systemctl status unifai.service
```

3. Environment Configuration Breakdown (.env)

Every operational parameter must be declared in `/opt/unifai/.env` (or project root `.env`). Live secrets must never be committed to git.

```
# ===== # SECTION 1: PUBLIC DOMAIN
& NETWORK ADDRESSING # ===== # The
canonical public HTTPS address of your instance. Used for CORS, OAuth callbacks, # and generating Browser
Guard PAC discovery URLs. SERVER_DOMAIN=https://unifai.yourcompany.com # Listening port and interface for
the Go backend APP_PORT=6000 APP_HOST=0.0.0.0 # Storage directory for runtime configs, logs, and attachments
APP_DIR=/app/data # Container and Network Proxy names CONTAINER_NAME=raksha_tech PROXY_PORT=18182
NETWORK_PROXY_CONTAINER_NAME=raksha_browser_ai_proxy DOCKER_NETWORK=raksha-network #
===== # SECTION 2: DATABASE
PERSISTENCE (POSTGRESQL) # =====
DB_TYPE=postgres DB_HOST=127.0.0.1 DB_PORT=5432 DB_NAME=unifai_db DB_USER=unifai_user
DB_PASSWORD=SuperStrongDBPassword_ReplaceInProduction! # Set to 'require' or 'verify-full' in production
when Postgres has TLS enabled DB_SSL_MODE=disable #
===== # SECTION 3: INITIAL
BOOTSTRAP ADMIN CREDENTIALS # =====
# Used ONLY to seed the initial root administrator on first boot. ADMIN_EMAIL=security-admin@yourcompany.com
ADMIN_PASSWORD=ChangeMeImmediatelyAfterFirstLogin_987#@! #
===== # SECTION 4: CRYPTOGRAPHY,
SECRETS & INTEGRITY (CRITICAL) #
===== # Master encryption key for
database fields at rest (AES-256-GCM). # Generate with: openssl rand -base64 32
UNIFAI_ENCRYPTION_KEY=w3dF9kJ+z1bT5L0k8Q9w2e4r6t8y0u2i4o6p8a0s2d4= # Fleet-wide secret shared between
Browser Guard agents and server. # Generate with: openssl rand -base64 32
RAKSHA_GUARD_SECRET=G8kL0mN2p4Q6r8S0t2V4w6X8y0A2b4C6d8E0f2G4h6I= # When 1: Enforces fail-closed rejection if
guard secret is missing or invalid. RAKSHA_GUARD_REQUIRE_SECRET=1 # Anti PAC-hijacking flag. Prevents clients
from injecting arbitrary proxy targets. RAKSHA_PAC_ALLOW_QUERY_PROXY=0 # Secret for signing password-reset
tokens (HMAC-SHA256, minimum 32 chars). # Generate with: openssl rand -base64 48
PASSWORD_RESET_SECRET=J9kL2mN4p6Q8r0S2t4V6w8X0y2A4b6C8d0E2f4G6h8I0j2K4l6M8n0P2q4R6s8T0= # Multi-node cluster
replication secret (when deploying behind a load balancer)
CLUSTER_REPLICATE_SECRET=K1mN3p5Q7r9S1t3V5w7X9y1A3b5C7d9E1f3G5h7I9j1K3l5M7n9P1q3R5s7T9u1= #
===== # SECTION 5: REVERSE PROXY &
SECURITY HEADERS # ===== # Enable
X-Forwarded-Proto and X-Forwarded-For evaluation from trusted reverse proxy TRUST_PROXY_HEADERS=1 #
Comma-separated list of trusted upstream proxy CIDRs (e.g. 127.0.0.1, 10.0.0.0/8)
TRUSTED_PROXIES=127.0.0.1,172.16.0.0/12 #
===== # SECTION 6: OPTIONAL AI
GUARD BOT (OLLAMA) & BRANDING #
=====
OLLAMA_URL=http://127.0.0.1:11434 RAKSHA_COMPANY_NAME=Your Company Name
RAKSHA_COMPANY_LOGO=/your-company-logo.png # Logging format and level LOG_LEVEL=info LOG_STYLE=json
```

4. Reverse Proxy Setup (Nginx & Caddy Production Configurations)

The Go backend must run behind a production-hardened reverse proxy terminating TLS, managing HTTP/2, enforcing security headers, and handling WebSocket upgrades.

4.1 Production Nginx Configuration

Create `/etc/nginx/sites-available/unifai.conf`:

```
# Upstream definition for UnifAI Go backend upstream unifai_backend { server 127.0.0.1:6000; keepalive 64; }
# Redirect all plain HTTP traffic to HTTPS server { listen 80; listen [::]:80; server_name
unifai.yourcompany.com; # Let's Encrypt challenge path location /.well-known/acme-challenge/ { root
/var/www/certbot; } location / { return 301 https://$host$request_uri; } } # Primary HTTPS Production Server
server { listen 443 ssl http2; listen [::]:443 ssl http2; server_name unifai.yourcompany.com; # TLS
Certificate paths (Let's Encrypt or Custom CA) ssl_certificate
/etc/letsencrypt/live/unifai.yourcompany.com/fullchain.pem; ssl_certificate_key
/etc/letsencrypt/live/unifai.yourcompany.com/privkey.pem; # Modern TLS cipher suite ssl_protocols TLSv1.2
TLSv1.3; ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-AES256-GCM-SHA384
:ECDHE-RSA-AES256-GCM-SHA384:DHE-RSA-AES128-GCM-SHA256:DHE-RSA-AES256-GCM-SHA384; ssl_prefer_server_ciphers
off; ssl_session_timeout 1d; ssl_session_cache shared:SSL:10m; ssl_session_tickets off; # Security Headers
add_header X-Content-Type-Options nosniff always; add_header X-Frame-Options SAMEORIGIN always; add_header
X-XSS-Protection "1; mode=block" always; add_header Strict-Transport-Security "max-age=31536000;
includeSubDomains; preload" always; add_header Referrer-Policy "strict-origin-when-cross-origin" always; #
Maximum payload size for file uploads (documents, images, guard packages) client_max_body_size 100M;
client_body_buffer_size 128k; # Proxy buffer settings for fast streaming responses (SSE) proxy_buffering off;
proxy_cache off; proxy_read_timeout 600s; proxy_send_timeout 600s; proxy_connect_timeout 60s; # Root proxy
location location / { proxy_pass http://unifai_backend; proxy_http_version 1.1; # WebSocket upgrade headers
proxy_set_header Upgrade $http_upgrade; proxy_set_header Connection $connection_upgrade; # Standard client
forwarding headers proxy_set_header Host $host; proxy_set_header X-Real-IP $remote_addr; proxy_set_header
X-Forwarded-For $proxy_add_x_forwarded_for; proxy_set_header X-Forwarded-Proto https; proxy_set_header
X-Forwarded-Host $host; proxy_set_header X-Forwarded-Port 443; } # Browser Guard download and installer
assets caching location /api/browser-ai/download/ { proxy_pass http://unifai_backend; proxy_http_version
1.1; proxy_set_header Host $host; proxy_set_header X-Forwarded-Proto https; add_header Cache-Control
"private, no-cache, no-store, must-revalidate"; } } # Required for WebSocket connection upgrades in Nginx
http context # Add this inside /etc/nginx/nginx.conf if not already present: # map $http_upgrade
$connection_upgrade { # default upgrade; # '' close; # }
```

Enable configuration and test Nginx:

```
sudo ln -s /etc/nginx/sites-available/unifai.conf /etc/nginx/sites-enabled/ sudo nginx -t sudo systemctl
reload nginx
```

4.2 Alternative: Caddyfile Configuration (Zero-Config HTTPS)

If using Caddy as the reverse proxy:

```
unifai.yourcompany.com { reverse_proxy 127.0.0.1:6000 { header_up Host {host} header_up X-Real-IP
{remote_host} header_up X-Forwarded-Proto https transport http { read_buffer 0 write_buffer 0 } } }
```

5. Operations, Health Monitoring & Upgrades

5.1 Service Health Verification

Test the service endpoints using **curl**:

```
# Gateway Health Check curl -v http://127.0.0.1:6000/health # Expected HTTP 200 OK:
{"status":"ok","time":"..."} # Prometheus Metrics Check (if token enabled) curl -v -H "Authorization: Bearer
<RAKSHA_METRICS_TOKEN>" http://127.0.0.1:6000/metrics
```

5.2 Log Management with **journalctl** & Logrotate

When using systemd, view real-time logs with:

```
# Follow real-time server output sudo journalctl -u unifai.service -f # Filter for errors or fatal events
sudo journalctl -u unifai.service -p err..emerg -n 100
```

5.3 Zero-Downtime Rolling Upgrades

To upgrade the binary without dropping active client connections:

```
# 1. Download or compile the new binary sudo cp unifai-server-v2.5.0 /opt/unifai/bin/unifai-server.new sudo
chmod 755 /opt/unifai/bin/unifai-server.new # 2. Swap binary atomically sudo mv
/opt/unifai/bin/unifai-server.new /opt/unifai/bin/unifai-server # 3. Restart service with minimal latency
sudo systemctl restart unifai.service # 4. Verify startup sudo journalctl -u unifai.service -n 50 --no-pager
```

End of Server Implementation Guide.

UnifAI / Raksha Enterprise Administrator Guide

Complete Admin Console Manual: Configuration, Governance & Fleet Management

Document Version: 2.4.0

Classification: Enterprise Administration & Security Governance

Target Audience: Platform Administrators, InfoSec Teams, Compliance Officers, Workspace Managers

1. Initial Access & First-Run Security Checklist

Upon launching the UnifAI platform, follow this mandatory security onboarding sequence before inviting users or onboarding endpoints:

1. Initial Root Sign-In:

- Navigate to your instance: <https://unifai.yourcompany.com/auth/login>.
- Authenticate using the bootstrap credentials defined in your server environment (**ADMIN_EMAIL** and **ADMIN_PASSWORD**).

2. Immediate Password Rotation:

- Click your profile avatar in the top-right corner \rightarrow **Account Settings** \rightarrow **Security**.
- Change the password immediately to a strong, high-entropy passphrase (minimum 16 characters).

3. Dedicated Admin Account Provisioning:

- Navigate to **Governance** \rightarrow **Users**.
- Create a named personal administrator account with the **admin** role.
- Demote or securely store the bootstrap account for break-glass emergency use only.

4. SMTP / Email Notification Setup:

- Navigate to **Settings** \rightarrow **SMTP Configuration**.
- Provide your corporate SMTP gateway credentials (host, port 587/465, TLS, user, password, from-address).
- Send a test email to verify that registration verification codes, password resets, and high-severity compliance alerts function correctly.

5. Branding & Legal Notice Customization:

- Under **Settings** \rightarrow **General Settings**, update the Company Name, Portal Logo, and End-User Terms of Use notice shown upon login.
-

2. Governance, User Lifecycle & RBAC Administration

2.1 User Lifecycle Management

Located at **Governance** \rightarrow **Users** (</workspace/governance/users>):

Operation	Steps in Admin UI	Underlying API Route
Approve Pending User	Select user row in Pending tab \rightrightarrow Click Approve . Assign Role and initial Allowed Sections.	POST /api/session/users/{id}/approve
Reject Sign-Up	Select user row \rightrightarrow Click Reject . Enter optional explanation reason.	POST /api/session/users/{id}/reject
Direct User Creation	Click + Create User . Enter Email, Full Name, Initial Password, Role, Budget, and Rate Limit.	POST /api/session/users
Role Assignment	Open user profile \rightrightarrow Select Role dropdown (admin , sub_admin , user , or Custom Role).	PUT /api/users/{id}/role
Section-Scoped Access	Under Allowed Sections , toggle specific sidebar items (e.g. Prompt Repository, Logs, Browser AI).	PUT /api/session/users/{id}
Account Deactivation	Click user actions menu \rightrightarrow Disable Account . Immediately revokes all active sessions.	DELETE /api/session/users/{id}

2.2 Roles & Permissions Engine (RBAC)

Located at **Governance** \rightrightarrow **Roles & Permissions** (**/workspace/governance/roles**):

UnifAI features a granular enterprise authorization matrix based on:

- **33 System Resources:** **GuardrailsConfig, GuardrailsProviders, GuardrailRules, UserProvisioning, Cluster, Settings, Users, Logs, Observability, Dashboard, VirtualKeys, ModelProvider, Plugins, MCPGateway, MCPToolGroups, MCPLogs, AdaptiveRouter, AuditLogs, Customers, Teams, RBAC, Governance, RoutingRules, PromptRepository, PromptDeploymentStrategy, SkillsRepository, AccessProfiles, APIKeys, Inference, Metrics, FeatureFlags, CircuitBreaker.**
- **6 Discrete Operations:** **Create, Read, Update, Delete, View, Download.**
- **Data Access Control (DAC):**
 - **all-data:** Role grants visibility across the entire enterprise tenant.
 - **own-data:** Restricts user visibility strictly to records, keys, or logs authored by the authenticated user.

Creating a Custom Role:

1. Click + **Create Custom Role**.
2. Name the role (e.g., **ComplianceAuditor, PromptEngineerLead**).
3. Set the Data Access Control scope (**all-data** or **own-data**).
4. Select the specific resource checkboxes and assign allowable operations.
5. Click **Save Role**. Custom roles immediately appear in user assignment menus.

2.3 Access Profiles (Policy Templates)

Located at **Governance** \rightrightarrow **Access Profiles** (**/workspace/governance/access-profiles**):

Access Profiles allow you to bundle permissions, model allowances, rate limits, and allowed sections into reusable, versioned policy packages.

- **Creating a Profile:** Click + **New Access Profile**, enter Name (e.g., **Contractor-Limited-Profile**), Description, and assign specific AI models allowed for inference.

- **Binding to Users:** When onboarding dozens of employees, simply select the Access Profile to automatically synchronize limits and permissions.

3. Model Providers & Model Catalog

Located at **Models** **Model Providers** (</workspace/models/providers>) and **Model Catalog**:

3.1 Connecting Commercial & Private AI Providers

UnifAI supports over 28 providers out of the box:

```
##### UnifAI Model Catalog #####  
#####  
▼ ▼ ▼ ▼ ▼ ##### OpenAI  
■ Anthropic ■ Gemini ■ Bedrock ■ Ollama ■ gpt-4o ■ claude-3.5 ■ 1.5-pro ■ claude-3 ■  
llama3.3 ■ o1, o3 ■ sonnet ■ 2.0-flash ■ titan ■ deepseek ■
```

Step-by-Step Provider Connection:

1. Navigate to **Models**  **Model Providers**.
2. Click **+ Add Provider** and choose the vendor (e.g., OpenAI, Anthropic, Google Gemini, Azure OpenAI, Ollama).
3. Fill in the connection parameters:
 - **Provider Name:** Unique identifier (e.g., **openai-production-tier1**).
 - **API Key:** Third-party vendor API secret (stored with AES-256-GCM encryption).
 - **Base URL (Optional):** Custom proxy URL or private Azure/Ollama endpoint (e.g. **http://ollama-server:11434**).
 - **Weight / Priority:** Integer weight used for automatic weighted load balancing.
4. Click **Test Connection**. The gateway will execute a lightweight health probe to verify credentials.
5. Click **Save Provider**. Available models will automatically synchronize into the **Model Catalog**.

3.2 Fallback Cascades & Adaptive Routing

Configure fallback chains to guarantee 99.999% availability:

- If **gpt-4o** returns an upstream HTTP 500/503 or hits a rate limit, the gateway automatically cascades the request to **claude-3-5-sonnet** or an internal Ollama model with zero client code changes.

4. Virtual Keys & Spend Governance

Located at **Virtual Keys** (/workspace/virtual-keys):

Virtual Keys are the gateway's primary credential mechanism for client applications, developers, and internal systems. They completely shield upstream provider API keys from exposure.

4.1 Creating a Virtual Key

1. Click + **Create Virtual Key**.
2. Provide key parameters:
 - **Key Name:** Identifier (e.g., **customer-support-agent-prod**).

- **Workspace / Team:** Assign to a department or project.
 - **Allowed Models:** Restrict the key to specific models (e.g., only **gpt-4o-mini** and **claude-3-haiku**).
 - **Budget Limit ():** Monthly or total hard dollar spend cap (e.g., **500.00**).
 - **Rate Limits:**
 - Requests Per Minute (RPM) cap (e.g., **120 RPM**).
 - Tokens Per Minute (TPM) cap (e.g., **100,000 TPM**).
3. Click **Generate Key**.
 4. **Copy the generated key immediately.** The secret (**vk-live-...**) is displayed only once.

5. Browser AI / Guard Fleet Management

Located at **Browser AI** (**/workspace/browser-ai/***):

The Browser AI suite allows administrators to manage, monitor, and enforce corporate security policies across all employee web interactions with public AI platforms.

Browser AI Admin Menu ■■■ Target Websites → Configure which AI sites are inspected ■■■ Guard Rules → Define DLP, regex, keywords, and AI Guard Bot rules ■■■ Prompt Logs → Full audit trail of prompts, blocks, and redactions ■■■ Guard Agents → Fleet health, OS versions, and agent status ■■■ Security Insights → Visual analytics on top violations and risky prompts ■■■ Setup & Download → Build, package, and distribute Windows/macOS installers

5.1 Configuring Target Websites

Navigate to **Browser AI** \rightrightarrow **Target Websites**:

1. Click **+ Add Target Website**.
2. Enter the domain or regex pattern:
 - Example 1: **chatgpt.com** (Protocol: HTTPS, Target Name: "OpenAI ChatGPT")
 - Example 2: **claude.ai** (Target Name: "Anthropic Claude")
 - Example 3: **perplexity.ai** (Target Name: "Perplexity AI")
3. Toggle **Enabled** to **Active**.
4. When saved, all connected endpoint guards immediately receive the updated PAC and proxy routing list on their next heartbeat.

5.2 Creating Guard Rules (DLP & Guard Bot)

Navigate to **Browser AI** \rightrightarrow **Guard Rules**:

1. Click **+ Add Rule**.
2. Define Rule Criteria:
 - **Rule Name:** e.g., **Block Credit Card Numbers (PCI-DSS)**.
 - **Target Scope:** Select All Websites or specific target platforms.
 - **Matching Engine:**
 - **Regex:** e.g. **\b(?:4[0-9]{12}(?:[0-9]{3})?|5[1-5][0-9]{14}|3[47][0-9]{13})\b**
 - **Keywords:** Comma-separated list (e.g. **CONFIDENTIAL_PROJECT_APOLLO, merger_terms**).
 - **AI Guard Bot:** Semantic assessment prompt evaluated by local Ollama LLM.

- **Action:**
 - **BLOCK:** Drop the prompt and display an in-browser compliance violation notification.
 - **REDACT:** Mask the sensitive substring with **[REDACTED_PCI_DATA]** and let the prompt proceed.
 - **WARN:** Allow the prompt but flag an alert in security logs and trigger an admin webhook.
3. Set **Priority** (1 – 100, where 1 is highest priority).
 4. Click **Save Rule**.

5.3 Guard Agent Fleet Monitoring

Navigate to **Browser AI**  **Guard Agents**:

- View the active fleet of connected laptops:
- Agent ID, Employee Username, Hostname, Operating System (Windows 11, macOS Sequoia).
- Guard Version (e.g. **v1.1.15**).
- Heartbeat status: **Online** (green indicator, active within 60s) or **Offline**.
- Last synced policy revision.

5.4 Rebuilding & Packaging Guard Installers

Navigate to **Browser AI**  **Setup**:

1. View the current distribution packages for **Windows (EXE)** and **macOS (PKG/ZIP)**.
2. If you updated the server domain or fleet secret, click **Rebuild & Publish Packages**.
3. Download the standalone installers directly or copy the one-line deployment script to distribute via MDM (Microsoft Intune, Jamf Pro, Kandji).
4. **Company Uninstall Key:** Set the mandatory master secret required if an employee or technician attempts to uninstall the guard client.

6. Workspace Configuration Views

UnifAI provides seven dedicated configuration views located under **Workspace**  **Config** (**/workspace/config/***):

6.1 Client Settings (**/workspace/config/client-settings**)

- Configures default timeouts, client retry counts, backoff multipliers, and streaming chunk buffer sizes for all outbound gateway connections.

6.2 Compatibility (**/workspace/config/compatibility**)

- Manages legacy OpenAI API version compatibility layers, Anthropic message formatting adapters, and custom vendor header rewriting rules.

6.3 Caching (**/workspace/config/caching**)

- Configures the In-Memory, Redis, or Vector-based Semantic Cache.
- Set cache TTL (Time to Live in seconds), maximum memory allocation, and similarity thresholds (e.g. **0.92**) for semantic hit matching.

6.4 Security (/workspace/config/security)

- Enforces session expiration timers (default: 24 hours), MFA policies, brute-force lockout thresholds (maximum failed attempts and lockout window in minutes), and trusted proxy CIDRs.

6.5 API Keys (/workspace/config/api-keys)

- Enterprise gate for managing root master API access, key rotation schedules, and third-party credential vaults.

6.6 Performance Tuning (/workspace/config/performance-tuning)

- Fine-tunes fasthttp worker pool size, maximum concurrent connections, TCP keep-alive intervals, and read/write buffer allocations.

6.7 Feature Flags (/workspace/config/feature-flags)

- Toggles runtime system capabilities dynamically without restarting containers:
- **enable_mcp_gateway**: Activates Model Context Protocol proxying.
- **enable_semantic_cache**: Activates vector-based caching engine.
- **enable_ai_guard_bot**: Enables Ollama-powered semantic guardrails.
- **enforce_strict_dlp**: Switches guard fleet to hard fail-closed enforcement.

7. Audit Logging & Security Forensics

Located at **Observability** → **Audit Logs** (/workspace/observability/audit-logs) and **Browser AI** → **Prompt Logs**:

- Every mutating administrative action (key creation, user approval, rule change, provider update) is permanently recorded with timestamp, actor email, source IP address, and JSON diff of changes.
- Prompt logs record every intercepted employee prompt, match scores, triggered DLP rules, and redaction diffs for compliance auditing (HIPAA, SOC 2, ISO 27001, GDPR).

End of Administrator Guide.

UnifAI / Raksha Enterprise End-User & Developer Guide

Comprehensive User Handbook: Portal, API Integration & Browser Guard

Document Version: 2.4.0

Classification: Enterprise End-User Documentation

Target Audience: Employees, Software Developers, AI Engineers, Data Scientists, Knowledge Workers

1. Welcome to UnifAI & Raksha Enterprise

Your organization provides UnifAI to enable secure, productive, and compliant use of modern Generative Artificial Intelligence. The platform consists of two primary touchpoints:

- UnifAI Workspace (Web Portal):** An intuitive web application where you can explore approved AI models, test prompts in the playground, maintain your personal and team prompt repositories, discover specialized AI skills, and view your usage metrics.
 - Raksha Browser Guard (Desktop Agent):** A secure background utility installed on your work laptop that safeguards company confidential data, source code, and customer privacy while you interact with web-based AI tools (such as ChatGPT, Claude, Gemini, Perplexity, and DeepSeek).
-

2. Getting Started: Account Lifecycle

2.1 Registration & Sign-Up

- Open your corporate portal URL in your web browser (e.g., <https://unifai.yourcompany.com>).
- Click **Create Account** or **Sign Up**.
- Enter your corporate email address (e.g., jane.doe@yourcompany.com), full name, and select a secure password.
- Email Verification:** A 6-digit confirmation code will be sent to your inbox. Enter the code on screen to verify your email address.
- Admin Approval:** Once verified, your account enters the **Pending Approval** queue. Your IT or security team will approve your account and assign your initial role and permissions. You will receive an email confirmation once approved.

2.2 Signing In & Session Security

- Log in with your email and password.
- Enterprise sessions remain active for **24 hours** by default.
- If Multi-Factor Authentication (MFA) is enabled by your administrator, enter your one-time authenticator passcode when prompted.

2.3 Password Recovery

If you forget your password:

1. Click **Forgot Password** on the login screen.
2. Enter your registered email address.
3. Check your email for a time-sensitive verification code (valid for 10 minutes).
4. Enter the code and set your new password.

3. Navigating the Workspace Portal

Once authenticated, your sidebar displays the features assigned to your user profile:

Workspace Navigation ■■■ Playground → Chat directly with authorized enterprise AI models ■■■ Prompt Repository → Store, version, test, and share high-performing prompts ■■■ Skills Repository → Pre-built agent skills and workflow automations ■■■ Virtual Keys → View and manage developer API keys (if authorized) ■■■ Observability → Track your personal token usage, latency, and costs ■■■ Settings → Manage profile details, notifications, and theme

3.1 Model Playground

The Playground provides a clean, interactive chat interface to interact with company-approved LLMs without requiring personal accounts or credit cards:

- Select an active model from the top dropdown (e.g., **gpt-4o**, **claude-3-5-sonnet**, **gemini-1.5-pro**, **llama-3.3-70b**).
- Adjust parameters: Temperature (creativity), Max Tokens (response length), and System Instructions.
- Supports multi-turn conversations, code syntax highlighting, and instant copying of outputs.

3.2 Prompt Repository

Save time by cataloging reusable prompt templates for yourself and your team:

- Click **+ New Prompt**.
- Define your prompt template using variables (e.g., **Summarize the following customer feedback into 3 bullet points: {{customer_text}}**).
- Tag with categories: **Engineering**, **Marketing**, **Legal**, **Support**.
- Share with your department or keep private in your personal library.

3.3 Observability & Usage Analytics

Review your personal AI consumption:

- Total requests sent over the past 24 hours, 7 days, or 30 days.
- Total tokens consumed (input vs. output tokens).
- Estimated cost and latency metrics.

4. Developer Integration Guide (Virtual Keys & APIs)

Software engineers can integrate corporate AI capabilities directly into scripts, backend services, CI/CD pipelines, and internal tools using **Virtual Keys**.

4.1 Authentication & Endpoint Discovery

- All API requests are authenticated via standard Bearer tokens in the HTTP Authorization header:

Authorization: Bearer <YOUR_VIRTUAL_KEY>

- The gateway is 100% wire-compatible with the OpenAI REST specification.

4.2 Standard `curl` Request Example

```
curl https://unifai.yourcompany.com/v1/chat/completions \
-H "Authorization: Bearer vk-live-abcdef1234567890abcdef1234567890" \
-H "Content-Type: application/json" \
-d '{ "model": "gpt-4o", "messages": [ { "role": "system", "content": "You are a helpful software engineering assistant." }, { "role": "user", "content": "Write a Python function to parse JSON lines." } ], "temperature": 0.2 }'
```

4.3 Python Integration (Official OpenAI SDK)

Simply point the **base_url** to your company's UnifAI gateway:

```
import os from openai import OpenAI # Initialize client pointing to enterprise gateway client = OpenAI(
api_key=os.environ.get("UNIFAI_VIRTUAL_KEY", "vk-live-your-key-here"),
base_url="https://unifai.yourcompany.com/v1" ) # Standard chat completion call response =
client.chat.completions.create( model="claude-3-5-sonnet", # Use Anthropic, OpenAI, or Gemini
interchangeably! messages=[ { "role": "system", "content": "You are an enterprise AI assistant." }, { "role":
"user", "content": "Explain microservices architecture in 3 bullet points." } ], temperature=0.3,
max_tokens=300 ) print(response.choices[0].message.content)
```

4.4 Node.js / TypeScript Integration

```
import OpenAI from "openai"; const openai = new OpenAI({ apiKey: process.env.UNIFAI_VIRTUAL_KEY ||
"vk-live-your-key-here", baseUrl: "https://unifai.yourcompany.com/v1", }); async function main() { const
completion = await openai.chat.completions.create({ model: "gpt-4o-mini", messages: [{ role: "user",
content: "Generate a TypeScript interface for a User profile." }], });
console.log(completion.choices[0].message.content); } main();
```

4.5 LangChain & LiteLLM Integration

```
from langchain_openai import ChatOpenAI llm = ChatOpenAI( model="gpt-4o", api_key="vk-live-your-key-here",
base_url="https://unifai.yourcompany.com/v1" ) response = llm.invoke("What are the key benefits of vector
embeddings?") print(response.content)
```

4.6 IDE Integration (Cursor, VS Code Cline / Continue)

To route your AI coding tools through the secure corporate gateway:

- **Cursor Settings**  **Models:**
 - Override OpenAI Base URL: **https://unifai.yourcompany.com/v1**
 - Enter your Virtual Key in the OpenAI API Key field.
- **VS Code Continue / Cline Plugin:**
 - Set provider to **openai**.
 - Set **apiBase**: **https://unifai.yourcompany.com/v1**
 - Set **apiKey**: **vk-live-your-key-here**

5. Raksha Browser Guard: Employee Desktop Guide

5.1 What is Raksha Browser Guard?

Raksha Browser Guard is an endpoint security utility installed on your company computer. It runs silently in the background and acts as an intelligent safety shield when you visit public generative AI websites (ChatGPT, Claude.ai, Gemini, etc.).

5.2 Privacy Notice: What is Monitored vs. Ignored

- **Strictly Scoped:** The guard **ONLY** inspects traffic destined for configured AI websites (e.g. **chatgpt.com**, **claude.ai**).
- **Complete Privacy for All Other Traffic:** Your general web browsing, personal banking, social media, corporate email, and internal intranet sites **completely bypass the guard** and are never intercepted or viewed.

Employee Laptop Web Traffic ■■■■ General Browsing (Google, News, Banks) ■■■■ DIRECT TO INTERNET (Zero Interception) ■■■■ Corporate Intranet / Slack / Email ■■■■ DIRECT TO INTERNET (Zero Interception) ■■■■ Generative AI Websites (ChatGPT, etc.) ■■■■ RAKSHA GUARD (Local Safety Inspection) ■■■■
 ■■■■
 Blocked

5.3 Windows Installation Guide

1. Obtain the installer (**Raksha_Guard_Setup.exe**) from your IT portal or downloads section.
2. Double-click the installer and follow the on-screen wizard:
 - Accept the license agreement.
 - Click **Install** (Administrative elevation prompt may appear).
3. Once installation completes, the guard launches automatically.
4. **Verification:** Look for the **Raksha shield icon** in your Windows System Tray (near the clock). A green indicator indicates the guard is active and protecting your sessions.

5.4 macOS Installation Guide

1. Download the macOS installer package (**Raksha_Guard_macOS.pkg** or ZIP).
2. Open the package and complete the guided installation.
3. When prompted, allow the installer to trust the local security certificate in your macOS System Keychain.
4. **Verification:** The Raksha shield icon will appear in the macOS top menu bar.

5.5 In-Browser Experience & What You Will See

Event	What Happens	What You See in Browser
Normal Prompt	Prompt contains no confidential or restricted data.	Prompt sends immediately to ChatGPT/Claude as normal.
Redacted Prompt	Prompt contains sensitive data like a credit card number or internal IP address.	The sensitive text is automatically masked with [REDACTED_PCI_DATA] before reaching the AI.
Blocked Prompt	Prompt violates enterprise policy (e.g., contains proprietary source code or confidential merger documents).	The request is stopped. An in-browser modal explains: <i>"Prompt blocked by corporate security policy (Rule: Proprietary Code Protection)"</i> .

5.6 Troubleshooting & Helpdesk

- **Status is Offline:** Right-click the system tray icon and select **Restart Guard**.
- **Certificate Warning in Browser:** If Chrome or Edge shows an SSL warning on ChatGPT, open the tray icon and select **Reinstall Certificates**, then restart your browser.
- **Support Contact:** Reach out to your internal IT Helpdesk or email your security administrators at the contact address listed in your portal.

End of End-User & Developer Guide.

UnifAI / Raksha Enterprise Database Configuration Guide

Complete Persistence Architecture, Optimization, Security & Disaster Recovery

Document Version: 2.4.0

Classification: Enterprise Database Administration & Data Architecture

Target Audience: Database Administrators (DBA), DevOps Engineers, Systems Architects

1. Persistence Architecture & Database Engine Support

UnifAI utilizes GORM as its Object-Relational Mapping (ORM) layer, supporting three primary relational database engines:

Engine	Production Recommended	Minimum Version	Typical Use Case	High Availability Support
PostgreSQL	YES (Preferred)	14.x / 15.x / 16.x	Enterprise production, multi-tenant fleet, high concurrency.	Streaming replication, Patroni, PgBouncer
MySQL / MariaDB	Compatible	8.0+ / 10.6+	Secondary choice for MySQL-exclusive infrastructures.	Group Replication, Galera Cluster
SQLite	NO (Dev only)	3.40+	Local unit tests and developer laptop sandbox (sqlite_static).	Single-process file lock (No clustering)

2. Production PostgreSQL Step-by-Step Installation

2.1 Installing PostgreSQL 16 on Ubuntu 22.04 / 24.04 LTS

Execute the following commands on your database server:

```
# Add official PostgreSQL APT repository
sudo apt update && sudo apt install -y curl ca-certificates gnupg
lsb-release sudo install -d /etc/apt/keyrings curl -fsSL https://www.postgresql.org/media/keys/ACCC4CF8.asc
| sudo gpg --dearmor -o /etc/apt/keyrings/postgresql.gpg echo "deb
[signed-by=/etc/apt/keyrings/postgresql.gpg] http://apt.postgresql.org/pub/repos/apt $(lsb_release -cs)-pgdg
main" | sudo tee /etc/apt/sources.list.d/pgdg.list # Install PostgreSQL 16 server and client tools
sudo apt update sudo apt install -y postgresql-16 postgresql-contrib-16 # Verify installation and service status
sudo systemctl enable postgresql sudo systemctl status postgresql
```

2.2 Provisioning User, Database & Permissions

Log in as the **postgres** system user and execute the provisioning commands:

```
sudo -u postgres psql
```

Execute SQL commands within the **psql** console:

```
-- 1. Create dedicated application user with SCRAM-SHA-256 password hashing CREATE USER unifai_user WITH
PASSWORD 'StrongEnterpriseDBPassword_ReplaceInProd!_2026'; -- 2. Create production database with UTF-8
encoding CREATE DATABASE unifai_db WITH OWNER unifai_user ENCODING 'UTF8' LC_COLLATE 'en_US.UTF-8' LC_CTYPE
'en_US.UTF-8'; -- 3. Connect to the database \c unifai_db -- 4. Grant schema privileges GRANT ALL PRIVILEGES
ON DATABASE unifai_db TO unifai_user; GRANT ALL ON SCHEMA public TO unifai_user; ALTER DEFAULT PRIVILEGES IN
SCHEMA public GRANT ALL ON TABLES TO unifai_user; ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT ALL ON
SEQUENCES TO unifai_user; -- 5. Enable optional performance and UUID extensions CREATE EXTENSION IF NOT
EXISTS "uuid-oss"; CREATE EXTENSION IF NOT EXISTS "pg_trgm"; CREATE EXTENSION IF NOT EXISTS "btree_gist"; --
Exit psql \q
```

3. Network Authentication & Security Configuration

3.1 pg_hba.conf Authentication Rules

Edit `/etc/postgresql/16/main/pg_hba.conf` to enforce modern **scram-sha-256** password authentication and restrict network access strictly to the application server:

```
# TYPE DATABASE USER ADDRESS METHOD # Local administrative access via UNIX domain socket local all postgres
peer local all all scram-sha-256 # IPv4 local loopback connections (if backend runs on same host) host
unifai_db unifai_user 127.0.0.1/32 scram-sha-256 # Dedicated Application Server Subnet (if backend is on a
separate host/container) host unifai_db unifai_user 172.16.0.0/16 scram-sha-256 host unifai_db unifai_user
10.0.0.0/8 scram-sha-256 # Reject all other unauthorized external traffic host all all 0.0.0.0/0 reject
```

3.2 Binding & Listening Address (postgresql.conf)

Edit `/etc/postgresql/16/main/postgresql.conf`:

```
# Listen only on loopback and internal private VPC interface listen_addresses = 'localhost, 172.16.10.5' port
= 5432 max_connections = 300
```

Reload PostgreSQL to apply configuration changes:

```
sudo systemctl reload postgresql
```

4. Enabling SSL/TLS Encryption for Database Connections

For enterprise compliance, all database traffic across networks must be encrypted in transit.

4.1 Generating PostgreSQL Server Certificates

```
sudo mkdir -p /etc/postgresql/ssl cd /etc/postgresql/ssl # Generate private key for PostgreSQL server sudo
openssl genrsa -out server.key 4096 sudo chmod 600 server.key # Generate self-signed certificate (or issue
via corporate CA) sudo openssl req -new -x509 -days 3650 -key server.key -out server.crt -subj
"/CN=db.internal.yourcompany.com" sudo chmod 644 server.crt sudo chown -R postgres:postgres
/etc/postgresql/ssl
```

4.2 Configuring PostgreSQL for SSL

In `/etc/postgresql/16/main/postgresql.conf`:

```
ssl = on ssl_cert_file = '/etc/postgresql/ssl/server.crt' ssl_key_file = '/etc/postgresql/ssl/server.key'
ssl_min_protocol_version = 'TLSv1.2' ssl_ciphers = 'HIGH:!aNULL:!MD5'
```

4.3 Updating Application `.env` for SSL Mode

In your UnifAI `.env` file, configure:

```
DB_TYPE=postgres DB_HOST=db.internal.yourcompany.com DB_PORT=5432 DB_NAME=unifai_db DB_USER=unifai_user
DB_PASSWORD=StrongEnterpriseDBPassword_ReplaceInProd!_2026 # SSL Modes: 'disable', 'require', 'verify-ca',
or 'verify-full' DB_SSL_MODE=require
```

5. Schema & Table Architecture

UnifAI organizes its database tables into two core logical domains managed via GORM auto-migration:

5.1 ConfigStore Domain (System State & Policies)

- **users**: User profiles, hashed passwords, roles, status (**pending**, **approved**, **disabled**), spending limits.
- **roles**, **permissions**, **resource_catalog**: Granular RBAC definitions (33 resources × 6 operations).
- **access_profiles**: Reusable permission and quota templates.
- **model_providers**: Third-party vendor credentials (encrypted at rest), weights, health status.
- **virtual_keys**: Developer tokens, token budgets, rate limits, allowed model lists.
- **target_websites**: URLs, domains, and regex patterns intercepted by the Browser Guard fleet.
- **guard_rules**: Real-time DLP patterns, regexes, keywords, and AI Guard Bot policies.
- **system_settings**: Global configurations, SMTP settings, branding, security thresholds.

5.2 LogsStore Domain (Observability & Forensics)

- **inference_logs**: Records model ID, input tokens, output tokens, latency, cost, virtual key ID, and user ID.
- **browser_ai_audit_logs**: Records intercepted employee prompts, target websites, matched DLP rules, redaction diffs, and blocking actions.
- **audit_logs**: Mutation logs capturing administrative actions for compliance reporting.

6. PostgreSQL Performance Tuning (**postgresql.conf**)

Tune these parameters based on your database host RAM:

```
# ===== # OPTIMIZED FOR 16 GB RAM
DEDICATED DATABASE SERVER # ===== #
Memory settings shared_buffers = 4GB # 25% of total RAM effective_cache_size = 12GB # 75% of total RAM
maintenance_work_mem = 1GB work_mem = 16MB # Per query sort memory wal_buffers = 16MB # Checkpoints and Write
Ahead Log (WAL) min_wal_size = 1GB max_wal_size = 16GB checkpoint_completion_target = 0.9 checkpoint_timeout
= 15min # Query Planner Optimization random_page_cost = 1.1 # Fast SSD / NVMe storage
effective_io_concurrency = 200 # Concurrent disk IO operations # Background Writer bgwriter_delay = 20ms
bgwriter_lru_maxpages = 100 bgwriter_lru_multiplier = 2.0
```

7. Connection Pooling with PgBouncer

For installations exceeding 500 concurrent users or multiple gateway instances, deploy PgBouncer to eliminate connection establishment overhead:

Install PgBouncer:

```
sudo apt install -y pgbouncer
```

Configure **/etc/pgbouncer/pgbouncer.ini**:

```
[databases] unifai_db = host=127.0.0.1 port=5432 dbname=unifai_db [pgbouncer] logfile =
/var/log/postgresql/pgbouncer.log pidfile = /var/run/postgresql/pgbouncer.pid listen_addr = 127.0.0.1
listen_port = 6432 auth_type = scram-sha-256 auth_file = /etc/pgbouncer/userlist.txt pool_mode = transaction
max_client_conn = 1000 default_pool_size = 50 min_pool_size = 10 reserve_pool_size = 5 reserve_pool_timeout =
5
```

Update your **.env** to point to port **6432**:

```
DB_PORT=6432
```

8. Automated Backup & Disaster Recovery

8.1 On-Demand Backup Commands (pg_dump)

```
# 1. Custom binary compressed backup (Recommended for restores) pg_dump -h 127.0.0.1 -U unifai_user -Fc -b -v
-f "unifai_db_$(date +%Y%m%d_%H%M%S).dump" unifai_db # 2. Plain SQL text backup (Gzip compressed) pg_dump -h
127.0.0.1 -U unifai_user -d unifai_db | gzip > "unifai_db_$(date +%Y%m%d_%H%M%S).sql.gz"
```

8.2 Production Automated Daily Backup Script

Save as **/opt/unifai/scripts/db_backup.sh** and make executable (**chmod +x**):

```
#!/bin/bash # ===== # UnifAI
Production PostgreSQL Automated Daily Backup Script #
===== set -e
BACKUP_DIR="/var/backups/unifai" DATE=$(date +%Y%m%d_%H%M%S)
BACKUP_FILE="${BACKUP_DIR}/unifai_db_${DATE}.dump" LOG_FILE="/var/log/unifai_db_backup.log"
RETENTION_DAYS=14 mkdir -p "${BACKUP_DIR}" echo "[$(date)] Starting UnifAI PostgreSQL backup..." >>
"${LOG_FILE}" # Execute pg_dump using credentials from ~/.pgpass or environment export
PGPASSWORD="StrongEnterpriseDBPassword_ReplaceInProd!_2026" pg_dump -h 127.0.0.1 -p 5432 -U unifai_user -Fc
-b -v unifai_db > "${BACKUP_FILE}" 2>> "${LOG_FILE}" if [ $? -eq 0 ]; then echo "[$(date)] Backup successful:
${BACKUP_FILE} ($(du -h ${BACKUP_FILE} | cut -f1))" >> "${LOG_FILE}" else echo "[$(date)] ERROR: Backup
failed!" >> "${LOG_FILE}" exit 1 fi # Clean up backups older than retention window find "${BACKUP_DIR}" -name
"unifai_db_*.dump" -type f -mtime +${RETENTION_DAYS} -delete echo "[$(date)] Cleaned up backups older than
${RETENTION_DAYS} days." >> "${LOG_FILE}"
```

Schedule via Crontab to run every midnight at 02:00 AM:

```
# Add to root crontab (crontab -e) 0 2 * * * /opt/unifai/scripts/db_backup.sh >/dev/null 2>&1
```

8.3 Disaster Recovery: Complete Database Restore Procedure

To restore the database from a backup file:

```
# 1. Terminate any active connections to the database sudo -u postgres psql -c "SELECT
pg_terminate_backend(pid) FROM pg_stat_activity WHERE datname = 'unifai_db' AND pid <> pg_backend_pid();" #
2. Drop and recreate database sudo -u postgres psql -c "DROP DATABASE IF EXISTS unifai_db;" sudo -u postgres
psql -c "CREATE DATABASE unifai_db WITH OWNER unifai_user ENCODING 'UTF8';" # 3. Restore schema and data
using pg_restore sudo -u postgres pg_restore -d unifai_db -v -O -x
"/var/backups/unifai/unifai_db_20261003_120000.dump" # 4. Verify table count and integrity sudo -u postgres
psql -d unifai_db -c "\dt"
```

End of Database Configuration Guide.

Complete Container Operations, Command Handbook, SSL Certificate Generation & Trust Store Integration

Target Audience: DevOps Engineers, Cloud Engineers, SREs, Security Operations (SecOps)

```
# ===== # 1. PREREQUISITE: CREATE
EXTERNAL DOCKER NETWORKS # ===== #
Create the external Ollama network if not already present docker network create 1panel-network # Create the
primary application bridge network (if custom name specified) docker network create raksha-network # Verify
created networks docker network ls #
===== # 2. STARTING CONTAINERS #
===== # Start the primary UnifAI
container in the background (detached mode) docker compose up -d # Start both primary backend AND the
optional network proxy container docker compose --profile network-proxy up -d # Start containers with rebuild
of images (forced re-compilation) docker compose up -d --build # Start containers without using cache during
build docker compose build --no-cache && docker compose up -d #
===== # 3. STATUS & INSPECTION
COMMANDS # ===== # Check status of
running containers, ports, and health docker compose ps # Show status including stopped or exited containers
docker compose ps -a # View live container resource consumption (CPU %, Memory, Network I/O, PIDs) docker
stats raksha_tech # Inspect detailed container JSON metadata (IP address, mounts, environment) docker inspect
raksha_tech # Inspect Docker network and see connected container IPs docker network inspect raksha-network
docker network inspect 1panel-network #
===== # 4. LOGS & REAL-TIME
MONITORING # ===== # Follow
real-time output of all containers docker compose logs -f # Follow logs of the primary Go backend with last
100 lines docker compose logs -f --tail=100 raksha_tech # Follow logs of the network proxy container docker
compose logs -f --tail=100 raksha_browser_ai_proxy # Search logs for errors or warnings docker compose logs
raksha_tech | grep -i "error" #
===== # 5. RESTARTING, STOPPING &
REMOVAL # ===== # Gracefully
restart the backend container (after editing .env) docker compose restart raksha_tech # Stop running
containers without removing them docker compose stop # Stop and remove containers, networks, and internal
links docker compose down # Stop and remove containers, networks, and anonymous volumes (CAUTION: does not
delete bind mounts) docker compose down -v #
===== # 6. EXECUTING COMMANDS
INSIDE RUNNING CONTAINERS # ===== #
Open an interactive shell inside the running backend container docker exec -it raksha_tech /bin/sh # (or bash
if available) docker exec -it raksha_tech /bin/bash # Execute a health check directly inside the container
docker exec -it raksha_tech curl -v http://localhost:6000/health # Check environment variables active inside
the container docker exec -it raksha_tech env | grep -E "APP_PORT|DB_HOST|SERVER_DOMAIN" # Inspect data
volume contents inside container docker exec -it raksha_tech ls -la /app/data #
===== # 7. CLEANUP & DISK PRUNING
COMMANDS # ===== # Remove unused
dangling images docker image prune -f # Comprehensive cleanup: remove stopped containers, unused networks,
and dangling images docker system prune -f # Full nuclear prune: removes all unused images, stopped
containers, and unused volumes (CAUTION) docker system prune -a --volumes -f
```

3. SSL/TLS Certificate Generation & Management

Production enterprise deployments require valid TLS certificates. This section provides exact commands for obtaining certificates via Let's Encrypt (Certbot), generating private self-signed certificates with OpenSSL, and generating the Root CA certificate for Browser Guard interception.

3.1 Obtaining Free Public SSL Certificates with Let's Encrypt (Certbot)

Certbot provides trusted certificates accepted by all modern operating systems and web browsers.

Prerequisites

- Your domain name (e.g. **unifai.yourcompany.com**) must have an active DNS **A** record pointing to your server's public IP address.
- Ports **80** and **443** must be open on your server firewall.

Method A: Standalone Mode (Before starting Nginx)

```
# Install Certbot sudo apt update && sudo apt install -y certbot # Request certificate in standalone mode
sudo certbot certonly --standalone \ --preferred-challenges http \ -d unifai.yourcompany.com \ --email
security-admin@yourcompany.com \ --agree-tos \ --no-eff-email # Certificates are saved to: # Certificate &
Full Chain: /etc/letsencrypt/live/unifai.yourcompany.com/fullchain.pem # Private Key:
/etc/letsencrypt/live/unifai.yourcompany.com/privkey.pem
```

Method B: Webroot Mode (With Nginx already running)

```
# Create certbot challenge directory sudo mkdir -p /var/www/certbot # Request certificate using webroot sudo
certbot certonly --webroot \ -w /var/www/certbot \ -d unifai.yourcompany.com \ --email
security-admin@yourcompany.com \ --agree-tos # Test automated certificate renewal sudo certbot renew
--dry-run
```

Method C: Automated Renewal Crontab

Certbot certificates expire after 90 days. Set up automatic renewal:

```
# Add cron job to renew twice daily and reload Nginx echo "0 3,15 * * * root certbot renew --quiet
--post-hook 'systemctl reload nginx'" | sudo tee -a /etc/crontab
```

3.2 Generating Self-Signed SSL/TLS Certificates with OpenSSL

For internal labs, air-gapped VPCs, or testing environments without public DNS:

Single-Command 4096-bit RSA Certificate with SAN (Subject Alternative Names)

```
# Create directory for custom certificates sudo mkdir -p /etc/ssl/unifai && cd /etc/ssl/unifai # Generate
private key and self-signed certificate valid for 3 years (1095 days) sudo openssl req -x509 -nodes -days
1095 -newkey rsa:4096 \ -keyout unifai_server.key \ -out unifai_server.crt \ -subj
"/C=US/ST=California/L=SanFrancisco/O=YourCompany/OU=IT/CN=unifai.yourcompany.com" \ -addext
"subjectAltName=DNS:unifai.yourcompany.com,DNS:localhost,IP:127.0.0.1" # Restrict private key permissions
sudo chmod 600 unifai_server.key sudo chmod 644 unifai_server.crt
```

Inspecting and Verifying the Generated Certificate

```
# Print certificate human-readable details, validity dates, and SAN extensions openssl x509 -in
unifai_server.crt -text -noout # Verify certificate matches private key modulus openssl x509 -noout -modulus
-in unifai_server.crt | openssl md5 openssl rsa -noout -modulus -in unifai_server.key | openssl md5 # Output
MD5 hashes MUST be identical!
```

3.3 Generating Root CA Certificate for Raksha Browser Guard (MITM Proxy)

To inspect HTTPS traffic between employee browsers and target Generative AI websites (ChatGPT, Claude, Gemini), Browser Guard requires an internal Root Certificate Authority (CA).

Step-by-Step Root CA Creation with OpenSSL

```
# Create CA authority directory mkdir -p /opt/unifai/certs/ca && cd /opt/unifai/certs/ca # 1. Generate Root CA Private Key (4096-bit RSA) openssl genrsa -out raksha-ca.key 4096 chmod 400 raksha-ca.key # 2. Generate Root CA Certificate (valid for 10 years / 3650 days) openssl req -x509 -new -nodes -key raksha-ca.key -sha256 -days 3650 \ -out raksha-ca.crt \ -subj "/C=US/ST=State/L=City/O=Raksha Security Enterprise/OU=Fleet Security/CN=Raksha Root CA - Enterprise AI Guard" # 3. Create combined PEM file for mitmproxy cat raksha-ca.key raksha-ca.crt > mitmproxy-ca.pem chmod 600 mitmproxy-ca.pem
```

3.4 Installing the Root CA Certificate into Client Operating Systems

For Browser Guard to inspect HTTPS sessions without browser security warnings (**ERR_CERT_AUTHORITY_INVALID**), the Root CA certificate (**raksha-ca.crt**) must be installed into the client trust store.

A. Windows Client Installation (Automated via CMD / PowerShell / Inno Setup)

Execute as **Administrator**:

```
:: Using built-in Windows certutil utility certutil -addstore -f "Root" "C:\path\to\raksha-ca.crt"
```

Verify installation in Windows Certificate Manager:

```
certutil -verifystore "Root" "Raksha Root CA - Enterprise AI Guard"
```

B. macOS Client Installation (Terminal / Jamf / Kandji)

Execute with **sudo**:

```
# Add certificate to macOS System Keychain and grant full trust for SSL/TLS sudo security add-trusted-cert \ -d \ -r trustRoot \ -k /Library/Keychains/System.keychain \ /path/to/raksha-ca.crt
```

Verify on macOS:

```
security find-certificate -c "Raksha Root CA - Enterprise AI Guard" /Library/Keychains/System.keychain
```

C. Linux (Ubuntu / Debian) Client Installation

```
# Copy certificate to trusted store directory sudo cp raksha-ca.crt /usr/local/share/ca-certificates/raksha-ca.crt # Update system certificate authorities sudo update-ca-certificates
```

D. Mozilla Firefox NSS Database Installation (Cross-Platform)

Firefox uses its own internal NSS certificate database rather than the OS trust store. To install into Firefox profiles via CLI:

```
# Install libnss3-tools sudo apt install -y libnss3-tools # Linux # or choco install nss -y # Windows # Import into all Firefox user profiles for certDB in $(find $HOME/.mozilla/firefox* -name "cert9.db"); do certdir=$(dirname ${certDB}) certutil -A -n "Raksha Root CA" -t "TCu,Cu,Tu" -i /path/to/raksha-ca.crt -d sql:${certdir} done
```

4. Reverse Proxy Integration: Connecting Nginx to Docker

When deploying via Docker Compose, Nginx runs on the host server (or in a dedicated edge container) and proxies incoming HTTPS requests to the Docker container port (**6000**).

4.1 Production Nginx Configuration File

Save to **/etc/nginx/sites-available/unifai:**

```
# Upstream pointing to Docker published port on localhost upstream docker_unifai_backend { server
127.0.0.1:6000; keepalive 32; } # Plain HTTP to HTTPS redirect server { listen 80; listen [::]:80;
server_name unifai.yourcompany.com; return 301 https://$host$request_uri; } # Production HTTPS Server server
{ listen 443 ssl http2; listen [::]:443 ssl http2; server_name unifai.yourcompany.com; # SSL Certificates
ssl_certificate /etc/letsencrypt/live/unifai.yourcompany.com/fullchain.pem; ssl_certificate_key
/etc/letsencrypt/live/unifai.yourcompany.com/privkey.pem; # Protocols and Ciphers ssl_protocols TLSv1.2
TLSv1.3; ssl_ciphers HIGH:!aNULL:!MD5; ssl_prefer_server_ciphers on; ssl_session_cache shared:SSL:10m;
ssl_session_timeout 1d; # Max upload limit for Guard installers and document embeddings client_max_body_size
100M; # Proxy to Docker container location / { proxy_pass http://docker_unifai_backend; proxy_http_version
1.1; # WebSocket support proxy_set_header Upgrade $http_upgrade; proxy_set_header Connection "upgrade"; #
Request forwarding headers proxy_set_header Host $host; proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for; proxy_set_header X-Forwarded-Proto https;
proxy_set_header X-Forwarded-Host $host; proxy_set_header X-Forwarded-Port 443; # Disable buffering for
real-time streaming AI responses (SSE) proxy_buffering off; proxy_read_timeout 600s; proxy_connect_timeout
60s; } }
```

Enable and reload Nginx:

```
sudo ln -sf /etc/nginx/sites-available/unifai /etc/nginx/sites-enabled/ sudo nginx -t && sudo systemctl
reload nginx
```

5. SSL/TLS Verification & Diagnostics Commands

Always test your SSL configuration and verify end-to-end handshake:

```
# 1. Test TLS Handshake and inspect presented certificate chain openssl s_client -connect
unifai.yourcompany.com:443 -servername unifai.yourcompany.com # 2. Check HTTP status, redirection, and
headers using curl curl -vI https://unifai.yourcompany.com/health # 3. Verify TLS 1.3 protocol negotiation
openssl s_client -connect unifai.yourcompany.com:443 -tls1_3 # 4. Check certificate expiration date directly
from remote server echo | openssl s_client -connect unifai.yourcompany.com:443 -servername
unifai.yourcompany.com 2>/dev/null | openssl x509 -noout -dates
```

End of Docker & SSL/TLS Configuration Guide.